

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ

ІМЕНІ ІВАНА ФРАНКА

Факультет прикладної математики та інформатики

(повне найменування назва факультету)

кафедра кібербезпеки

(повна назва кафедри)

КУРСОВА РОБОТА

на тему: «СТЕГАНОГРАФІЧНИЙ ЗАХИСТ АВТОРСЬКИХ ПРАВ:
ДОСЛІДЖЕННЯ МЕТОДІВ ВБУДОВУВАННЯ ЦИФРОВИХ ВОДЯНИХ
ЗНАКІВ ЗА ДОПОМОГОЮ ГЕНЕРАТИВНИХ НЕЙРОННИХ МЕРЕЖ»

Студента 5 курсу групи ПМКМ-11

напряму підготовки: «ІІІ в
кібербезпеці»

Оберемка І.С.

(прізвище та ініціали)

Керівник:

професор, доктор технічних наук

Пелешко Д.Д.

(посада, вчене звання, науковий
ступінь, прізвище та ініціали)

Національна шкала: _____

Кількість балів: _____ Оцінка: ECTS

ЗМІСТ

ЗМІСТ.....	2
ВСТУП.....	3
Розділ 1 ОГЛЯД НАЯВНОЇ ЛІТЕРАТУРИ НА ЗАДАНУ ТЕМУ.....	7
1.1 Опис технології вбудовування водяних знаків за допомогою стеганографії.....	7
1.1.1 Історичні передумови розвитку стеганографії.....	7
1.1.3 Цифрові водяні знаки як прикладний випадок стеганографії.....	11
1.2. Огляд існуючих алгоритмів для вбудовування водяних знаків за допомогою стеганографії.....	13
1.2.1. KGW.....	13
1.2.2. Алгоритм EXP (Aaronson, 2023).....	14
1.2.3. Unbiased Watermark (Hu et al., 2023).....	15
1.2.4. SEMSTAMP (Hou et al., 2024).....	16
1.2.5. SynthID-Text (Dathathri et al., 2024).....	16
1.2.6. SegmentWM (Qu et al., 2024).....	17
1.2.7. Публічно-детектовний водяний знак Fairuze (Fairuze et al., 2023)	18
1.3. Особливості та недоліки існуючих технологій вбудовування водяних знаків. Поточні виклики галузі.....	18
Розділ 2 РЕЗУЛЬТАТИ ЕКСПЕРИМЕНТІВ.....	20
2.1. Огляд метрик отриманих в результаті запуску алгоритмів вбудовування водяних знаків наявних у фреймворку MarkLLM.....	20
2.1.1. Методика експериментального дослідження.....	20
2.1.2. Аналіз результатів детектування на чистому тексті.....	22
2.1.3. Аналіз якості згенерованого тексту.....	24
2.1.4. Аналіз стійкості до атак.....	26
2.2. Розробка алгоритму стеганографічного вбудовування водяного знаку сумісного з технологією блокчейн на основі проведених досліджень.....	27
2.2.1. Поточна екосистема blockchain-native автономних агентів на основі стандарту ERC-8004.....	27
2.2.2. Критерії сумісності алгоритму вбудовування водяного знака з блокчейн-екосистемою.....	28
2.2.3. Дизайн алгоритму ChainMark: асиметричне сегментне вбудовування ECDSA-підпису.....	29
2.2.3.1. Алгоритм кодування.....	31
2.2.3.2. Алгоритм декодування.....	32
2.2.3.3. Властивості та сумісність із критеріями.....	33
ВИСНОВКИ.....	35
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	36

ВСТУП

Стрімкий розвиток великих мовних моделей (Large Language Models, LLM) та побудованих на їх основі автономних програмних агентів поступово перетворює концепцію «штучного інтелекту як сервісу» з предмету наукової фантастики на повноправного учасника цифрової економіки. Якщо ще декілька років тому автономні ШІ-агенти розглядалися переважно у науково-дослідному контексті, то станом на 2026 рік вже існує цілий клас так званих zero-human компаній, що функціонують без участі людини у своїх щоденних операціях [1], а також спеціалізована інфраструктура для розгортання та координації таких агентів [2]. Паралельно з цим формується новий шар платіжних протоколів, орієнтованих саме на потреби автономних агентів і побудованих на технології блокчейн, що дозволяє агентам безпосередньо здійснювати мікроплатежі, укладати смарт-контракти та обмінюватися цифровими активами без участі традиційних фінансових посередників.

Масштаби цього явища підтверджуються конкретними кількісними показниками. Згідно з даними публічних реєстрів [3; 4], у рамках стандарту ERC-8004 [5] вже зареєстровано понад 192 тисячі автономних агентів. Сукупний обсяг операцій у межах протоколу Agent Commerce Protocol (ACP) [6] наближається до 500 мільйонів доларів США, а кількість транзакцій у платіжному протоколі x402 [7] перевищує 170 мільйонів. Це свідчить про те, що автономні агенти вже сьогодні самостійно надають фінансові, аналітичні, креативні та консультаційні послуги, доставляють кінцевий результат замовнику й отримують винагороду в криптоактивах.

Водночас разом із зростанням обсягів автономної активності агентів загострюється фундаментальна проблема, яка лежить у площині кібербезпеки та інформаційної безпеки: на сьогодні не існує надійного, криптографічно обґрунтованого механізму, який би дозволяв публічно та однозначно довести, що конкретний фрагмент згенерованого текстового контенту був створений саме певним автономним агентом і саме певною мовною моделлю. Відсутність такого механізму породжує одразу декілька гострих ризиків: неможливість захисту авторських прав агента та його власника, уразливість до атак типу “impersonation”, труднощі з аудитом виконаних робіт у межах смарт-контрактів, а також складність боротьби з

масованою генерацією дезінформації від імені довірених джерел. Класичні методи цифрових підписів у цьому випадку не є достатніми, оскільки текстовий контент часто переформатовується, частково копіюється або інтегрується в інші документи, що руйнує жорстку прив'язку підпису до файлу.

Перспективним напрямом розв'язання цієї проблеми є застосування стеганографічних методів вбудовування цифрових водяних знаків безпосередньо у процесі генерації тексту мовною моделлю. На відміну від зовнішніх підписів, такий водяний знак стає невід'ємною частиною самого згенерованого контенту і зберігається навіть за умов часткового редагування. Однак переважна більшість існуючих рішень орієнтована на симетричну схему, у якій ключ генерації збігається з ключем верифікації, що принципово несумісно з відкритою моделлю довіри блокчейн-екосистем, де верифікація має бути доступною будь-якому учаснику мережі. Саме на закриття цієї прогалини й спрямована дана робота, що визначає її актуальність та доцільність для розвитку галузі кібербезпеки.

Особистий внесок автора полягає у систематизації та порівняльному аналізі сучасних методів стеганографічного вбудовування водяних знаків у текст, згенерований мовними моделями, у проведенні експериментального дослідження цих методів з позиції їх сумісності з блокчейн-інфраструктурою, а також у розробці власного дизайну алгоритму, орієнтованого на потреби публічної верифікації авторства автономних агентів. Референсна реалізація запропонованого алгоритму та повний експериментальний код роботи опубліковані у відкритому репозиторії [22].

Мета і завдання дослідження. Метою роботи є огляд та порівняльний аналіз існуючих методів стеганографічного вбудовування цифрових водяних знаків у текстовий контент, створений великими мовними моделями, а також розробка дизайну алгоритму стеганографічного вбудовування цифрових водяних знаків, сумісного з технологією блокчейн та придатного для застосування у задачах захисту авторських прав автономних ШІ-агентів.

Для досягнення поставленої мети необхідно розв'язати такі завдання:

1. Здійснити огляд існуючих алгоритмів стеганографічного вбудовування цифрових водяних знаків у текст, згенерований

великими мовними моделями, з виокремленням їх ключових властивостей, сильних та слабких сторін.

2. Провести експериментальне дослідження відібраних алгоритмів з метою предметного аналізу їх сумісності з технологією блокчейн, насамперед з огляду на можливість публічної верифікації водяного знаку у відкритій мережі.
3. Розробити дизайн алгоритму стеганографічного вбудовування цифрових водяних знаків, який задовольнятиме таким критеріям:
 - a. асиметричність схеми, тобто розмежування між приватним ключем генерації та публічним ключем верифікації, що забезпечує можливість публічної перевірки авторства без розкриття таємної інформації власника агента;
 - b. мультибітовість, тобто можливість вбудувати і відтворити після вбудовування масив бітів;
 - c. мінімальний статистичний біас згенерованого контенту, тобто збереження якості та природності тексту відносно немаркованого розподілу мовної моделі;
 - d. мінімальний вплив на швидкість генерації контенту, що є критично важливим для агентів, які працюють у режимі реального часу та обслуговують велику кількість запитів;
 - e. мінімальний обсяг тексту, для якого можлива достовірна публічна верифікація водяного знаку.

Об'єкт дослідження – процес стеганографічного захисту авторських прав на текстовий контент, створений великими мовними моделями та автономними програмними агентами, які функціонують у блокчейн-екосистемах.

Предмет дослідження – методи та алгоритми стеганографічного вбудовування цифрових водяних знаків у текст, згенерований великими мовними моделями, що допускають публічну криптографічну верифікацію та сумісні з технологією блокчейн.

Розділ 1 ОГЛЯД НА ЯВНОЇ ЛІТЕРАТУРИ НА ЗАДАНУ ТЕМУ

1.1 Опис технології вбудовування водяних знаків за допомогою стеганографії

1.1.1 Історичні передумови розвитку стеганографії

Термін «стеганографія» походить від грецьких слів *στεγανός* («прихований») та *γράφειν* («писати») і дослівно означає «приховане письмо». На відміну від криптографії, що приховує лише зміст переданого повідомлення, стеганографія має на меті приховати сам факт існування комунікації між сторонами [9]. Ця ідея виявилася настільки інтуїтивно зрозумілою та практично корисною, що сьогодні налічує понад 2500 років задокументованої історії застосування у військовій, дипломатичній і приватній сферах.

Одним із найбільш ранніх задокументованих прикладів є епізод, описаний Геродотом і датований V ст. до н. е., коли грецький тиран міста Мілет, Гістіей, прагнучи закликати Арістагора до повстання проти перського царя, поголив голову свого довіреного раба, витатуював послання на шкірі та відправив гінця у дорогу лише після того, як його волосся повністю відросло й приховало текст [9]. Інший класичний приклад відносять до 480 р. до н. е., коли грек Демарат попередив спартанців про підготовку Ксеркса до вторгнення: він зіскоблив віск з пари дерев'яних табличок, написав послання безпосередньо на дереві, після чого знов покрив поверхню воском – таблички виглядали порожніми та не викликали підозри у вартових [9].

Як окрема дисципліна стеганографія сформувалася значно пізніше – у працях німецького абата Йоганна Трітемія (1462–1516), а саме в його трактаті «*Steganographia*», від назви якого і походить сучасний термін. Цей твір, виданий латиною, зовнішньо являв собою оповідь про спілкування з духами, проте насправді містив одні з перших систематичних методів приховування повідомлень у літературному тексті; зашифрований третій том вдалося розшифрувати лише наприкінці XX ст. незалежно

дослідниками Т. Ернстом та Дж. Рідсом [9]. У ту ж добу шотландська королева Марія Стюарт використовувала комбінацію криптографії та стеганографії для листування зі своїми прибічниками – листи виносили з її ув'язнення у затичках пивних бочок [9]. Італійський математик Джироламо Кардано запропонував так звану «решітку Кардано» – аркуш паперу із вирізаними отворами, що накладався на нібито нейтральний текст і відкривав справжнє повідомлення.

У XX ст. стеганографічні методи широко застосовувалися протягом обох світових війн. Нацистська Німеччина систематично використовувала мікродоти – мікрофільмові зображення, виконані зі збільшенням понад 200 разів і зменшені до розмірів крапки на друкарській машинці, що дозволяло переховувати у тексті стандартного листа цілі сторінки технічної інформації; також застосовувалися невидимі чорнила та так звані null-шифри [9]. Один з найбільш відомих null-шифрів – повідомлення «Apparently neutral's protest is thoroughly discounted and ignored. Isman hard hit. Blockade issue affects pretext for embargo on by-products, ejecting suets and vegetable oils», у якому за другою літерою кожного слова прочитується справжній зміст «Pershing sails from NY June 1» [9]. Прем'єр-міністр Великої Британії М. Тетчер, прагнучи виявити джерело витоку засекречених документів, наказала запрограмувати свої текстові процесори таким чином, щоб ідентифікатор автора кодувався у незначних варіаціях ширини проміжків між словами [9].

Спільною ідеєю, що проходить крізь усі ці приклади, є застосування «носія» з природною надлишковістю – шкіри людини, поверхні таблички, тексту листа, фотографічного зображення – у якому додаткова інформація приховується з мінімальним порушенням статистичного розподілу самого носія. Саме ця ідея пізніше була формалізована в межах теорії складності та лежить в основі сучасних стеганографічних схем, у тому числі тих, що застосовуються до тексту, згенерованого великими мовними моделями.

1.1.2 Формальна модель стеганографічної системи

Першу повноцінну формалізацію задачі стеганографії на мові теорії складності було запропоновано у роботі Н. Хоппера, Дж. Лангфорда та Л. фон Ана 2002 року [8], де було сформульовано визначення стеганографічної системи як криптографічного примітиву та доведено, що з існування односторонніх функцій випливає існування стеганографічно

секретних протоколів. Саме ця модель і використовується нижче як абстрактний опис технології вбудовування водяних знаків.

В основу моделі покладено сценарій «в'язниці», вперше сформульований Г. Сіммонсом у 1983 р. та згодом формалізований у [8]. Розглядаються двоє ув'язнених – Аліса і Боб, які прагнуть узгодити план втечі через єдиний доступний їм публічний канал зв'язку, що цілодобово відслідковується наглядачем Вардом. Якщо Вард виявляє у повідомленні будь-яку зашифровану або підозрілу інформацію, обох ув'язнених буде ізольовано. Завдання стеганографії, на відміну від класичної криптографії, полягає не у приховуванні змісту повідомлень, а у приховуванні самого факту їх існування: повідомлення Аліси повинні бути обчислювально нерозрізнюваними з природним трафіком каналу.

Формально канал зв'язку визначається як розподіл $\mathcal{C}$ на послідовностях бітів, кожен з яких супроводжується монотонно зростаючою часовою міткою. Інтуїтивно, $\mathcal{C}$ моделює природне «безневинне» спілкування – електронні листи, фотографії, фрагменти тексту тощо. Для опрацювання повідомлень з огляду на історію передачі h вводиться умовний розподіл $\mathcal{C}_h^b$ – розподіл наступного блока довжиною b бітів за умови вже спостережуваної історії h . Стеганографічний протокол має право робити часткові вибірки з оракула $M(h)$, що породжує блоки за розподілом $\mathcal{C}_h^b$.

Ключовою кількісною характеристикою каналу є мінімальна ентропія – найгірша оцінка інформативності одного блоку:

$$H_{\infty}(\mathcal{C}) = \min_{x \in X} \log_2 \frac{1}{\Pr_{\mathcal{C}}[x]},$$

де X – множина всіх можливих значень, які може видати канал; $\Pr_{\mathcal{C}}[x]$ – ймовірність появи конкретного значення x при витягуванні з розподілу $\mathcal{C}$;

Природна вимога $H_{\infty}(\mathcal{C}_h^b) > 1$ для будь-якої історії h забезпечує існування у каналі достатнього обсягу випадковості, у який можна вбудувати приховане повідомлення без помітного спотворення розподілу.

Стеганографічна система (стегосистема) визначається як пара ймовірнісних поліноміальних алгоритмів $\mathcal{S} = (SE, SD)$. Алгоритм кодування $SE^M(K, m, h)$ приймає на вхід секретний ключ $K \in \{0, 1\}^k$, приховуваний текст $m \in \{0, 1\}^*$, історію h та оракул M і повертає

послідовність блоків $c_1 \| c_2 \| \dots \| c_l$ (стеготекст), що становлять “обгорткове” повідомлення для передачі через канал. Алгоритм декодування $SD^M(K, c, h)$ за тим самим ключем відновлює прихований текст. Накладається умова коректності, що гарантує мінімальну ймовірність успішного декодування:

$$\Pr [SD^M(K, SE^M(K, m, h), h) = m] \geq \frac{2}{3}$$

Така порогова ймовірність є рішенням, що охоплює широкий клас стегосистем; її надалі доводять до близької до одиниці за допомогою кодів з виправленням помилок.

Стеганографічна секретність моделюється як гра між стегосистемою S та пасивним наглядачем W , який отримує доступ до оракула каналу M та одного з двох тестових оракулів: або до справжнього кодувальника $SE(K, \cdot, \cdot)$, або до фіктивного оракула $O(\cdot, \cdot)$, що ігнорує своє повідомлення-вхід і повертає чисту вибірку з каналу відповідної довжини. Перевага наглядача формалізується як модуль різниці ймовірностей його правильного відгадування:

$$\text{Adv}_{S,c}^{ss}(W) = \left| \Pr [W^{M, SE(K, \cdot, \cdot)} = 1] - \Pr [W^{M, O(\cdot, \cdot)} = 1] \right|$$

Стегосистема S називається (t, q, l, ε) -стеганографічно секретною проти атаки на обраний приховуваний текст (t, q, l, ε) , якщо для будь-якого наглядача W , що працює час t і робить q запитів сумарною довжиною l бітів, виконується нерівність $\text{Adv}_{S,c}^{ss}(W) \leq \varepsilon$ [8].

Інакше кажучи, секретна стегосистема – це така, поведінку якої жоден поліноміально обмежений противник не вирізнить від звичайного трафіку каналу з імовірністю помітно більшою за випадкове вгадування.

Базовою конструкцією, що задовольняє цьому означенню, є схема на основі псевдовипадкової функції $F_K: \{0, 1\}^d \times \{0, 1\}^b \rightarrow \{0, 1\}$ та процедури відбраковування семплування (rejection sampling). Кодувальник представляє приховане повідомлення як послідовність бітів $m = m_1 \| m_2 \| \dots \| m_l$ та для кожного біта m_i повторно семплює блок $c_i \leftarrow M(h)$ доти, доки не виконається рівність $F_K(N, c_i) = m_i$ або не вичерпано ліміт ітерацій (де N – спільний з декодером лічильник). Декодер відновлює прихований біт як $m_i = F_K(N, c_i)$.

У роботі [8] також формалізовано робастність – здатність стегосистеми чинити опір активному наглядачеві, що має право обмежено модифікувати стеготекст. Допустимі модифікації описуються відношенням $R \subseteq X \times Y$, де $(c, c') \in R$ означає, що блок c' є допустимим спотворенням блока c . Вводиться функція ймовірності маскування:

$$O(R, \mathcal{D}) = \max_y \sum_{(x', y) \in R} \Pr_{\mathcal{D}}[x'],$$

що задає верхню межу ймовірності, з якою активний наглядач може успішно «розчинити» свою модифікацію в природному розподілі $\mathcal{D}$. Пара $(R, \mathcal{C})$ називається δ -допустимою, якщо $O(R, \mathcal{C}_h^b) \leq \delta$ для будь-якої історії h , і саме за цієї умови у [8] доведено існування стегосистеми, що є одночасно секретною та робастною.

Ключова теорема цієї лінії досліджень встановлює, що небезпека стеганографічної конструкції на основі F_K обмежена згори небезпекою самої функції F_K як псевдовипадкової:

$$\text{InSec}_{\mathcal{S}, \mathcal{C}}^{ss}(t, q, l) \leq \text{InSec}_F^{prf}(t + O(\ell(l)), \ell(l)),$$

де $\ell(l)$ – функція розширення коду з виправленням помилок. Звідси випливає принциповий висновок: за умови існування односторонніх функцій (з яких будуються псевдовипадкові функції) існують і стеганографічно секретні протоколи [8]. Саме це теоретичне твердження є фундаментом, на якому пізніше були побудовані сучасні watermark-схеми для виходу великих мовних моделей: канал $\mathcal{C}$ у цьому випадку моделюється розподілом наступного токена за попереднім контекстом, а роль псевдовипадкової функції відіграє криптографічно стійка хеш-функція над контекстним вікном.

1.1.3 Цифрові водяні знаки як прикладний випадок стеганографії

Цифровий водяний знак – це окремий, прикладний випадок стеганографічного вбудовування, у якому приховуване повідомлення m є криптографічним маркером авторства, а каналом-носієм виступає сам згенерований цифровий контент: зображення, аудіо, відео або текст [9]. Ключова відмінність задачі вбудовування водяного знака від загальної

стеганографії полягає у тому, що тут зазвичай не потрібно передавати довільне повідомлення – достатньо, щоб з носія можна було надійно відтворити фіксований ідентифікатор автора або довести належність контенту певній особі чи моделі-генератору.

У літературі прийнято класифікувати цифрові водяні знаки за трьома ортогональними ознаками [9]: за робастністю – на крихкі, що руйнуються від найменшого редагування і використовуються для контролю цілісності, та робастні, що зберігаються при типових перетвореннях носія; за помітністю – на видимі та невидимі; за моделлю довіри – на приватні і публічні.

Саме остання дихотомія є визначальною для цієї курсової роботи: приватні водяні знаки передбачають, що ключ верифікації збігається з ключем генерації або похідний від нього і має бути відомий лише довіреному перевіряючому, тоді як публічні припускають верифікацію будь-яким стороннім учасником без розкриття секретного матеріалу автора. У контексті блокчейн-екосистем та автономних агентів практично прийнятними є виключно публічні схеми, що відповідає сформульованому у вступі критерію асиметричності.

З технічного боку методи вбудовування цифрових водяних знаків поділяються на дві великі групи [9]. Методи просторової області (image domain) маніпулюють безпосередньо значеннями пікселів або семплів – типовим представником є модифікація молодшого значущого біта (least significant bit, LSB). Методи частотної області (transform domain) виконують вбудовування у спектральних коефіцієнтах сигналу і застосовують для цього дискретне косинусне перетворення (discrete cosine transform, DCT) або вейвлет-перетворення, що забезпечує суттєво вищу робастність до типових атак. Для текстової стеганографії, що становить безпосередній інтерес у задачі захисту контенту LLM, у [9] виокремлено шість основних класів методів: відкритих проміжків, зміщення рядків, зміщення слів, синтаксичні, семантичні та ознакові. Перші три належать до форматних методів і легко руйнуються будь-яким переформатуванням; останні три оперують вже на рівні самої мови та є концептуально ближчими до сучасних watermark-схем для LLM, у яких вбудовування здійснюється на рівні розподілу ймовірностей наступного токена.

Не менш важливою стороною аналізу є модель атак на водяний знак. У [9] перелічено вісім типових класів атак: додавання шуму, фільтрація,

кадрування, стиснення, поворот і масштабування, статистичне усереднення, множинне вбудовування та атаки інших рівнів. Для текстового водяного знака аналогами цих атак є парафразування, переклад, скорочення, конкатенація з іншим текстом та атака усередненням за множиною згенерованих відповідей. робастність до цих атак безпосередньо корелює з критерієм робастності, що буде застосовано в експериментальному дослідженні наступних розділів.

Окремий внутрішній зв'язок існує між критеріями якості водяного знака і методами стеганоаналізу. Ще у 1993 р. К. Брауном було запропоновано базовий набір метрик для виявлення прихованих повідомлень у тексті: частота літер, частота слів, стискуваність, граматичний стиль і читабельність, семантична цілісність та контекстна узгодженість [9]. По суті, ці самі метрики формують зворотний бік критерію мінімального статистичного біасу, сформульованого у вступі: водяний знак, що проходить тест Брауна, є водночас і таким, що мінімально спотворює якість згенерованого тексту відносно еталонного розподілу мовної моделі.

Таким чином, класична стеганографія, формальна модель стегосистеми та сучасна задача вбудовування водяних знаків у текст великих мовних моделей складають єдиний концептуальний ланцюжок: від ідеї приховування самого факту комунікації – через її криптографічну формалізацію в моделі Хоппера – Лангфорда – фон Ана – до прикладних публічно-верифікованих схем для захисту авторських прав автономних агентів. Деталізований критичний огляд конкретних реалізацій останніх та їх порівняльний аналіз з точки зору сумісності з технологією блокчейн становить предмет наступних розділів цієї роботи.

1.2. Огляд існуючих алгоритмів для вбудовування водяних знаків за допомогою стеганографії

1.2.1. KGW

Алгоритм KGW [11] є фундаментальним зразком, на якому базується значна частина сучасних схем. Його ключова ідея полягає у поділі словника V на «зелений» список G та «червоний» список R у пропорції $\gamma : (1 - \gamma)$, причому конкретний поділ визначається псевдовипадково через хеш від попереднього токена s_{t-1} із секретним

ключем K . На етапі семплування до логітів усіх токенів зеленого списку додається фіксований зсув $\delta > 0$, що м'яко зміщує модель у бік генерації «зелених» токенів без жорсткої заборони низькоентропійних виборів:

$$\hat{p}_k^{(t)} = \frac{\exp(l_k^{(t)} + \delta \cdot \mathbf{1}[k \in G])}{\sum_i \exp(l_i^{(t)} + \delta \cdot \mathbf{1}[i \in G])},$$

де $\hat{p}_k^{(t)}$ – ймовірність вибору токена k на кроці t після вбудовування водяного знака, $l_k^{(t)}$ – початковий логіт LLM, δ – бонус для зелених токенів, $\mathbf{1}[k \in G]$ – індикатор належності токена до зеленого списку.

Детектування зводиться до підрахунку кількості зелених токенів $|s|_G$ у досліджуваному тексті довжиною T та проведення одно-пропорційного z-тесту проти нульової гіпотези про те, що текст не маркований:

$$z = \frac{|s|_G - \gamma T}{\sqrt{T\gamma(1 - \gamma)}},$$

де $|s|_G$ – кількість зелених токенів у тексті s ; T – загальна кількість токенів; γ – частка зеленого списку. Стандартні параметри $\gamma = 0,5$, $\delta = 2,0$, $z_{thr} = 4$ дають імовірність помилки першого роду (false positive) $3 \cdot 10^{-5}$ та виявляють водяний знак вже на ~ 25 токенах для тексту з помірною ентропією [11]. У термінах підрозділу 1.2 KGW реалізує симетричну стегосистему: один і той самий ключ K використовується і у SE (для генерації зелених списків), і у SD (для повторного відтворення цих списків при перевірці). Це принципово несумісно з блокчейн-моделлю довіри, у якій верифікатор не має доступу до приватного ключа автора.

1.2.2. Алгоритм EXP (Aaronson, 2023)

Алгоритм EXP [12], запропонований С. Аронсоном, реалізує концептуально іншу стратегію – він не модифікує розподіл p_θ , а лише змінює спосіб семплування з нього таким чином, щоб результуючий вибір токена був детермінованою функцією від розподілу та псевдовипадкової послідовності. Замість стандартного мультиноміального семплування алгоритм генерує для кожного токена вектор $u^{(t)} \in (0, 1)^{|V|}$ псевдовипадково із хешу попередніх h токенів та секретного ключа K , після чого вибирає токен як:

$$s^{(t)} = \arg \max_{k \in V} (u_k^{(t)})^{1/p_k^{(t)}},$$

де $s^{(t)}$ – токен, обраний на кроці t ; V – словник токенів LLM; k – індекс токена у словнику; $u_k^{(t)}$ – псевдовипадкове число для токена k на кроці t ; $p_k^{(t)}$ – немаркована ймовірність токена k за розподілом базової моделі.

Цей вираз є реалізацією Gumbel-max розподілу: якщо u_k розподілені рівномірно, то отриманий вибір збігається за розподілом з мультиноміальним семплуванням, тобто схема є distortion-free – маргінальний розподіл згенерованого тексту не відрізняється від еталонного. Детектування здійснюється через статистику:

$$S = \sum_{t=1}^T \ln(1/(1 - u_{s^{(t)}}^{(t)})),$$

математичне очікування якої для маркованого тексту перевищує T , тоді як для немаркованого дорівнює T [12]. У термінах формальної моделі ЕХР задовольняє вимогу нульового статистичного біасу, однак, як і KGW, є симетричною – детектор повинен володіти ключем K для повторного обчислення $u^{(t)}$.

1.2.3. Unbiased Watermark (Hu et al., 2023)

Метод Unbiased Watermark [13] узагальнює ідею Аронсона у вигляді сімейства так званих «функцій переважування». Замість прямого зміщення логітів алгоритм застосовує до розподілу $p^{(t)}$ перетворення F_K , що залежить від псевдовипадкового зерна, отриманого з хешу попередніх токенів та таємного ключа K :

$$\hat{p}_k^{(t)} = \frac{p_k^{(t)} \cdot F_K(k, c_t)}{\sum_j p_j^{(t)} \cdot F_K(j, c_t)},$$

де c_t – усе, що модель уже згенерувала до кроку t , а F_K – псевдовипадкова функція, яка трохи змінює ваги токенів. Ключова

властивість цього алгоритму: якщо усереднити результат за випадковим ключем K , то новий розподіл $\hat{p}_k^{(t)}$ у середньому збігається з початковим $p_k^{(t)}$, тобто метод не вносить систематичного викривлення (distortion-free). Детектування здійснюється через тест відношення правдоподібності (англ. log-likelihood ratio, LLR). Він порівнює, наскільки ймовірно побачити саме такий згенерований текст у випадку, коли водяний знак присутній ($\hat{p}^{(t)}$), проти випадку, коли його немає ($p^{(t)}$):

$$\text{LLR}(s_{1:T}) = \sum_{t=1}^T \ln \frac{\hat{p}_{s_t}^{(t)}}{p_{s_t}^{(t)}},$$

де $s_{1:T}$ – досліджуваний текст із T токенів; $\hat{p}_{s_t}^{(t)}$ та $p_{s_t}^{(t)}$ – ймовірності, які присвоюють обраному токenu s_t маркований і немаркований розподіли відповідно. Текст вважається маркованим, якщо отримане значення LLR настільки велике, що ймовірність випадково отримати таке для немаркованого тексту не перевищує $5 \cdot 10^{-4}$. Подібно до KGW і EXP, схема є симетричною. Перевагою для використання є відсутність будь-яких помітних статистичних відхилень тексту від еталонного, проте проблема секретності ключа залишається.

1.2.4. SEMSTAMP (Hou et al., 2024)

Алгоритм SEMSTAMP [14] переносить ідею «зеленого/червоного» списку KGW з рівня токенів на рівень речень, забезпечуючи робастність до парафразування – однієї з найбільш типових атак на LLM-водяні знаки. Кожне згенероване речення кодується у D -вимірне семантичне векторне представлення e_i за допомогою заздалегідь натренованого SBERT-енкодера, після чого простір вкладень розбивається псевдовипадковими гіперплощинами на 2^d комірок методом локально-чутливого хешування (LSH). Хеш від попереднього речення визначає підмножину «дозволених» комірок розміром $\gamma \cdot 2^d$. Алгоритм генерує речення-кандидати за допомогою rejection sampling до тих пір, поки SBERT-вкладення нового речення не потрапить у дозволену комірку, або не вичерпається ліміт спроб $N_{\max}$:

$$\text{LSH}(e_i) \in \{G(e_{i-1}, K) : |G| = \gamma \cdot 2^d\}$$

Детектор обчислює, скільки згенерованих речень потрапляють у відповідні «зелені» комірки, та проводить z-тест аналогічно KGW. Перевагою SEMSTAMP є робастність: оскільки парафразування зберігає семантичне вкладення близьким до оригіналу – водяний знак вціліває після парафразування. Платою є висока обчислювальна вартість – кожне речення генерується багаторазово до проходження тесту, що збільшує латентність генерації у 5–10 разів. Схема залишається симетричною.

1.2.5. SynthID-Text (Dathathri et al., 2024)

SynthID-Text [15], запропонований командою DeepMind та опублікований у Nature, базується на принципі турнірного семплування. Для кожного токена алгоритм генерує 2^L кандидатів незалежним семплуванням з $p^{(t)}$ та проводить серед них турнір з L раундів, у кожному з яких пара кандидатів змагається за «голоси», що отримуються з псевдовипадкових водяних функцій $g_\ell : V \times \mathbb{Z} \rightarrow \{0, 1\}$, $\ell = 1, \dots, L$. Псевдовипадковість визначається ключем K та контекстом (n -грамом попередніх tokenів). Маргінальний розподіл наступного токена не змінюється – водяний знак виявляється виключно через кореляцію між обраними токенами та значеннями g_ℓ . Детектування здійснюється шляхом обчислення середнього значення $g_\ell(s^{(t)}, \text{ctx})$ за всіма позиціями та порівняння з порогом 0,52 [15]. SynthID підтримує також байєсівський та зважений детектори. Перевагою є висока якість тексту й ефективна генерація; недоліком – симетричність схеми та вибагливість до якості семплування (потрібно багаторазове семплування для кожного токена).

1.2.6. SegmentWM (Qu et al., 2024)

SegmentWM [16] є першим у нашому переліку алгоритмом, що розширює задачу від zero-bit (виявлення факту маркування) до multi-bit (вбудовування корисного навантаження довжиною b біт). Повідомлення K розбивається на k сегментів довжиною b/k біт кожен, причому замість поділу tokenів на b груп (по одній на біт) використовується лише k груп, що різко зменшує дисбаланс у природній частоті tokenів. Для кожного токена i хеш від попереднього токена s_{i-1} задає номер сегмента $j_i \in \{0, \dots, k-1\}$, що цей токен має кодувати, а хеш пари (s_{i-1}, K_{j_i}) – псевдовипадковий «зелений» список для саме цієї пари (сегмент, попередній токен). Решта механізму збігається з KGW: до логітів зеленого

списку додається зсув δ . Декодер для кожного сегмента j перебирає всі $2^{b/k}$ можливих значень і обирає те, що максимізує кількість зелених токенів:

$$\hat{K}_j = \operatorname{argmax}_{v \in [0, 2^{b/k})} |\{i : j_i = j, s_i \in G(s_{i-1}, v)\}|$$

Для подолання помилок витягання, спричинених редагуванням тексту, поверх сегментної схеми застосовується код Ріда–Соломона $RS(n, k, t)$ з $t = \lfloor (n - k)/2 \rfloor$ помилок. У стандартній конфігурації MarkLLM використовуються $k = 64$ інформаційних та $n = 100$ кодових сегментів по 8 біт кожен, $\gamma = 0,25$. Складність вилучення дорівнює $O(k \cdot 2^{b/k})$, що дозволяє витягувати $\delta = 8,032$ -бітне повідомлення в 7-10 разів швидше ніж в інших мультибітових схемах [16]. У роботі [16] також доведено робастність до атак з обмеженою редакційною відстанню. Принципова перевага для блокчейн-сценарію – можливість вбудовувати мультибітове корисне навантаження. Однак сам механізм верифікації залишається симетричним: для перевірки потрібне знання ключа K та функцій формування зелених списків.

1.2.7. Публічно-детектовний водяний знак Fairoze (Fairoze et al., 2023)

Алгоритм Fairoze [17] є єдиною серед розглянутих схем, що задовольняє вимогу асиметричності, сформульовану у вступі. Ключова відмінність – використання асиметричної криптографії. Автор має приватний ключ підписання sk , з якого обчислюється підпис префіксної частини згенерованого тексту. Цей підпис далі вбудовується у наступну частину тексту посегментно: для кожного сегмента довжиною L_s символів обчислюється хеш-ланцюг $H_i = \text{SHA256}(s_{i-1} || i)$, i генерація продовжується до тих пір, поки b молодших біт цього хешу не співпадуть з відповідним фрагментом підпису σ . Детектор, маючи лише публічний ключ pk , повторно обчислює хеш-ланцюг, відновлює зашитий підпис і перевіряє його, збираючи σ з хешів молодших біт $H_1, H_2, \dots, H_n$. Для подолання потенційних помилок вбудовування використовується код Ріда–Соломона. Завдяки асиметричності схема Fairoze є природно сумісною з блокчейн-моделлю, в якій будь-який учасник мережі може верифікувати авторство через публічний ключ без розкриття секретів. Платою за цю властивість є значно більша мінімальна довжина тексту

(приблизно 520 біт сигнатури, що відповідає сотням токенів) та зменшена швидкість генерації через rejection sampling на рівні символів.

1.3. Особливості та недоліки існуючих технологій вбудовування водяних знаків. Поточні виклики галузі.

Узагальнюючи розглянуті алгоритми у термінах критеріїв, сформульованих у вступі, можна виокремити декілька закономірностей. По-перше, переважна більшість схем (KGW, EXP, Unbiased, SEMSTAMP, SynthID-Text, SegmentWM) є симетричними і тому принципово несумісні з відкритою блокчейн-моделлю довіри, у якій розкриття ключа знищує властивість невідрізнюваності водяного знака. По-друге, distortion-freeness у відомих реалізаціях (EXP, Unbiased, SynthID) досягається ціною уразливості до парафразування, оскільки сигнал кодується у тонкій кореляції між псевдовипадковими вибірками і токенами. По-третє, залежність між робастністю до парафразування та латентністю є монотонною: від KGW і EXP (<1 % накладних витрат, але руйнуються від найпростішого переписування) до SEMSTAMP і Fairoze (робастні, але у 5–10 разів повільніші через rejection sampling). По-четверте, єдиний публічно-верифікований алгоритм Fairoze має мінімальну довжину маркованого фрагмента порядку сотень токенів і суттєву залежність швидкості від ентропії моделі.

Жоден з розглянутих алгоритмів не задовольняє одночасно всім 5 критеріям; для сценарію автономних ШІ-агентів пріоритетними є асиметричність та помірна латентність, тоді як мінімальна довжина маркованого фрагмента обмежена знизу необхідністю умістити криптографічний підпис. Кількісне порівняння цих алгоритмів та обґрунтування підсумкового вибору базової схеми становить предмет наступного розділу роботи.

Розділ 2 РЕЗУЛЬТАТИ ЕКСПЕРИМЕНТІВ

2.1. Огляд метрик отриманих в результаті запуску алгоритмів вбудовування водяних знаків наявних у фреймворку MarkLLM

2.1.1. Методика експериментального дослідження

З метою кількісного порівняння розглянутих у розділі 2 алгоритмів за сформульованими у вступі критеріями було проведено серію експериментів на базі відкритого інструментарію MarkLLM [10]. Усі сім алгоритмів (KGW, EXP, Unbiased, SEMSTAMP, SynthID-Text, SegmentWM, Fairgoze) запускалися в однакових умовах: як базова мовна модель використовувалась Qwen2.5-Coder-1.5B у форматі вагів float16 на графічному процесорі NVIDIA з підтримкою обчислень за технологією CUDA, як джерело промптів – підмножина датасету C4 з набору MarkLLM, максимальна довжина продовження – 800 нових токенів, кількість незалежних запусків для кожної комбінації «алгоритм × сценарій» – $N = 20$. Як немаркований еталон використовувалися продовження C4, отримані тією ж моделлю без втручання у процес семплування. Експериментальний конвеєр складається з трьох послідовних етапів: на першому з промпта генерується пара текстів (маркований заданим алгоритмом і немаркований); на другому, опціональному, до маркованого тексту застосовується одна з атак; на третьому обидва тексти подаються у відповідний детектор разом із вимірюванням метрик якості.

Усі метрики, що використовуються у роботі, поділяються на три ортогональні класи – детектування, якості та робастності.

Метрики детектування характеризують здатність детектора відрізнити маркований текст від немаркованого і обчислюються за матрицею невідповідностей. Чутливість – частка коректно ідентифікованих маркованих текстів; специфічність $TNR = TN / (TN + FP)$ – частка коректно ідентифікованих немаркованих. Допоміжно вводяться $FPR = 1 - TNR$ (помилка першого

роду – хибне звинувачення немаркованого тексту у маркуванні) та $FNR = 1 - TPR$ (помилка другого роду – пропуск маркованого тексту).

Інтегральні $TPR = TP / (TP + FN)$ як гармонічне середнє точності та повноти і $ACC = (TP + TN) / (TP + TN + FP + FN)$ як частка правильних рішень. Поріг детектування для кожного алгоритму обирається з умови максимуму на пулі досліджуваних запусків.

Метрики якості оцінюють вартість вбудовування – наскільки сильно водяний знак спотворює згенерований текст порівняно з немаркованим базовим розподілом моделі. Перплексія:

$$PPL(s) = \exp \left(- \frac{1}{T} \sum_{t=1}^T \log p_{\theta}(s_t \mid s_{<t}) \right),$$

обчислена тією ж моделлю Qwen2.5-Coder-1.5B, інтерпретується саме як міра distortion-cost – статистичного відхилення стеготексту від природного розподілу LM. Log-diversity – логарифм відношення кількості унікальних n-грам до загальної їх кількості – характеризує лексичне різноманіття. Метрики BLEU та ROUGE-1, ROUGE-2, ROUGE-L вимірюють n-грамне перекриття стеготексту з референсним продовженням C4 (відповідно n-грамна точність, 1-грамне, 2-грамне покриття та найдовша спільна підпоследовність). BERTScore F1 із енкoderом roberta-large [18] обчислюється як косинусна близькість контекстуалізованих ембеддингів стеготексту й еталону і слугує найстійкішим до поверхневих перетворень показником збереження змісту.

Метрики робастності збігаються з метриками детектування, вимірними на тексті, попередньо спотвореному однією з чотирьох атак. Перша – парафразування за допомогою генеративної моделі Parrot, побудованої шляхом донавчання архітектури T5 на задачі перефразування [19], з шириною променевого пошуку 5. Друга – зворотний переклад за схемою англійська – китайська – англійська на базі дистильованої моделі машинного перекладу NLLB-200 з 600 млн параметрів [20]. Третя – контекстна заміна синонімів засобами маскованої мовної моделі BERT (base, uncased) [21] із часткою заміненних токенів 0,5. Четверта – випадкове видалення слів із імовірністю 0,3. Зниження F_1 детектора відносно «чистого» сценарію відображає чутливість конкретного алгоритму до відповідного класу спотворень і відповідає сформульованому у вступі критерію робастності.

2.1.2. Аналіз результатів детектування на чистому тексті

Результати детектування водяних знаків на немодифікованому маркованому тексті за умов експерименту, описаного у підрозділі 2.1.1, наведено у таблиці 2.1.2:

Таблиця 2.1.2. Результати детектування на чистому тексті

Алгоритм	TPR	TNR	FPR	FNR	F1	ACC	F1-best	Час, с
KGW	1.000	1.000	0.000	0.000	1.000	1.000	8.06	633.8
EXP	1.000	1.000	0.000	0.000	1.000	1.000	0.031	164.6
SynthID	1.000	1.000	0.000	0.000	1.000	1.000	0.528	660.0
SegmentWM	1.000	1.000	0.000	0.000	1.000	1.000	15.26	642.1
SEMSTAMP	1.000	1.000	0.000	0.000	1.000	1.000	4.18	718.4
Unbiased	1.000	1.000	0.000	0.000	1.000	1.000	$4.12 \cdot 10^{-4}$	905.2
Fairoze	1.000	1.000	0.000	0.000	1.000	1.000	0.618	1186.7

З наведених даних випливає, що на чистому, немодифікованому тексті всі сім розглянутих алгоритмів демонструють однаково ідеальну якість класифікації: $TPR = TNR = F_1 = ACC = 1,000$ за двадцяти незалежних запусків кожен. Це означає, що на горизонті 800 нових tokenів і за відсутності атак базова задача детектування – відрізнити маркований текст від немаркованого – не є дискримінаційною характеристикою серед сучасних схем: усі вони задовольняють перший необхідний критерій працездатності водяного знака, тобто наявність статистичного сигналу, який однозначно виявляється детектором за нульової частоти хибних спрацьовувань. Це узгоджується з теоретичним викладом підрозділу 1.2: для кожного з алгоритмів псевдовипадкова функція F_K задає достатньо різке статистичне зміщення, яке для текстів довжиною кількесот tokenів виходить далеко за межі флуктуацій немаркованого розподілу.

Водночас числове різноманіття обраних порогів та загальний час експерименту на ідентичному корпусі переконують у тому, що рівність метрик не означає рівноцінності алгоритмів. Фіксований час від запуску генерації до завершення детектування на одному й тому самому пулі

промптів коливається у межах від 164,6 с (EXP) до 1186,7 с (Fairoze), тобто понад семикратно, що вже на цьому етапі унаочнює критерій (в) з вступу – мінімальний вплив на швидкість генерації – і попередньо вказує на головний компроміс, який буде кількісно проаналізовано у подальших підрозділах: алгоритми з найбільш інформативним або криптографічно навантаженим сигналом (Fairoze, Unbiased, SEMSTAMP, SegmentWM) сплачують за це генерацією у 4–7 разів повільнішою за найдешевший EXP. Таким чином, отримана в цій точці експерименту еквівалентність детектування на чистому тексті переносить площину диференціації алгоритмів у дві ортогональні до неї площини – якості згенерованого тексту (підрозділ 3.3) та робастності до атак (підрозділ 3.4), які і дозволять у подальшому дати кінцеву кількісну відповідь на задачу вибору базової схеми для дизайну власного алгоритму.

2.1.3. Аналіз якості згенерованого тексту

Результати кількісної оцінки якості стеготексту отримана за методикою підрозділу 2.1.1, наведено у таблиці 2.1.3:

Таблиця 2.1.3. Результати перевірки якості згенерованого тексту

Алгоритм	PPL	Log-div.	BLEU	ROUGE-1	ROUGE-2	ROUGE-L	BERTScore F_1
–	10,22	6,88	1,02	0,205	0,034	0,103	0,8093
KGW	12,10	6,69	0,69	0,194	0,026	0,099	0,8029
EXP	5,29	5,36	2,79	0,301	0,048	0,162	0,8233
SynthID	5,25	6,08	1,03	0,192	0,038	0,106	0,8133
SEMSTAMP	11,73	7,08	0,90	0,198	0,031	0,097	0,8064
Unbiased	10,49	6,83	1,06	0,207	0,036	0,105	0,8116
SegmentWM	67,81	7,99	0,61	0,196	0,021	0,090	0,7922
Fairoze	18,44	6,81	0,65	0,191	0,025	0,092	0,8008

Аналіз отриманих значень дозволяє виокремити три якісно різних режими, у яких водяний знак впливає на текст. Перший – режим «нульового» статистичного біасу, у якому теоретична властивість distortion-freeness реалізується на практиці; до нього належить алгоритм Unbiased, для якого приріст перплексії становить лише $\Delta PPL = +0,27$, log-diversity 6,83 практично збігається з немаркованим еталоном 6,88, а BERTScore $F_1 = 0,8116$ навіть незначно перевищує референсний 0,8093 у межах статистичної похибки. Це повністю підтверджує висновки роботи [13] про те, що псевдовипадкова geweight-функція, усереднена за випадковим ключем, відтворює маргінальний розподіл наступного токена, і безпосередньо співвідноситься з критерієм (б) з вступу – мінімальним статистичним біасом маркованого тексту. До цього ж режиму, з невеликим запасом за PPL (+1.88 та +1.55 відповідно), належать KGW і SEMSTAMP, у яких BERTScore просідає лише на 0.006-0.003, що є практично невідчутним з огляду на те, що сама модель Qwen2.5-Coder-1.5B при

різних випадкових прогонах власних немаркованих продовжень дає аналогічного порядку розкид.

Другий режим – режим «зниженої» перплексії, парадоксальним чином характерний для distortion-free семплерів типу Gumbel-max (EXP та SynthID-Text). Обидва алгоритми показують , $PPL \approx 5,25 - 5,29$ що приблизно вдвічі менше за немарковану базову лінію 10,22, а їхні BLEU та ROUGE-L – навпаки, кратно вищі за всі інші схеми (2,79 та 0,162 для EXP проти 1,02 та 0,103 для еталона). На перший погляд це могло б свідчити про «покращення» якості, проте детальніший розгляд показує протилежне: знижена PPL досягається ціною концентрації семплера на найбільш імовірних токенах, що підтверджується одночасним падінням log-diversity до 5,36 та 6,08 – тобто текст робиться менш різноманітним, ближчим до моди розподілу, і саме тому формально ближчим до референсного продовження C4. Така інтерпретація узгоджується з міркуваннями підрозділу 2.2: обидві схеми не модифікують розподіл, а лише вибирають з нього детерміновано через функцію $\arg\max$, що для розподілів з вираженою концентрацією маси на топ-токенах систематично надає перевагу саме їм. У термінах критерію (б) ці алгоритми задовольняють вимогу мінімального біасу у статистичному сенсі (маргінальний розподіл за усередненням ключа збігається з еталоном), проте у термінах семантичного різноманіття – є більш консервативними, ніж сама мовна модель.

Третій режим – режим «зміщеного» розподілу, до якого належать SegmentWM та Fairoze. SegmentWM з його агресивним зсувом логітів $\delta = 8,0$ показує найвищу серед розглянутих схем перплексію 67,81 (у 6,6 разу більшу за немарковану базову лінію), а також найвище значення log-diversity 7,99, що відображає випадкове зміщення розподілу у бік менш імовірних токенів зеленого списку. Незважаючи на це, BERTScore $F_1 = 0,7922$ просідає лише на 0.017 відносно немаркованого тексту і на відносно KGW; цей факт є важливим методологічним зауваженням: висока перплексія вказує саме на сильне статистичне відхилення стеготексту від природного розподілу LM, а не на втрату змісту, оскільки контекстуалізовані ембеддинги RoBERTa-large продовжують збігатися з референсними у межах кількох тисячних. Алгоритм Fairoze посідає проміжне місце з BERTScore $F1 = 0.8008$ та $PPL = 18.44$ (+8.22 до еталона); це є очікуваною платою за rejection sampling на рівні символічних сегментів, який змушує модель повторно генерувати фрагменти до тих пір,

поки молодші біти SHA-256 хешу збігатимуться з відповідним фрагментом ECDSA-підпису.

Узагальнюючи цей блок експериментів у термінах критеріїв вступу, можна стверджувати, що жоден з розглянутих алгоритмів не призводить до катастрофічної втрати семантичної якості – розкид BERTScore F_1 між найкращим та найгіршим алгоритмом становить лише 0.031 (від 0,8233 у EXP до 0,7922 у SegmentWM), що значно менше за саму помилку обчислення BERTScore та засвідчує практичну застосовність усіх розглянутих схем.

2.1.4. Аналіз робастності до атак

Зведений показник F_1 детектора на текстах, попередньо спотворених за чотирма класами атак (підрозділ 2.1.1), наведено у таблиці 2.1.4:

Таблиця 2.1.4. Результати детекції на спотворених текстах

Алгоритм	Парафраз	Видалення слів	Контекстні синоніми	Зворотній переклад	Середнє
KGW	1.000	1.000	1.000	1.000	1.000
SEMSTAMP	1.000	0.976	1.000	0.950	0.982
EXP	1.000	0.976	1.000	0.842	0.955
SynthID	1.000	0.952	0.976	0.900	0.957
Unbiased	1.000	0.952	0.976	0.900	0.957
Fairoze	1.000	0.905	0.927	0.850	0.921
SegmentWM	1.000	1.000	0.644	0.947	0.898

Парафразування за моделлю Parrot не зачіпає жодний з детекторів – $F_1 = 1,000$ для всіх семи схем; з огляду на консервативний характер цього перефразатора (він орієнтований на NLU-задачі та зберігає довжину і лексичне ядро речень) ці значення слід трактувати як робастність до помірною парафразування, а не як універсальну робастність до парафразу взагалі. Найжорсткішою атакою виявився зворотний переклад через китайську: F_1 опускається до 0,842 у EXP та до 0,850 у Fairoze, що відповідає природі їх сигналу – Gumbel-max-кореляція з псевдовипадковою змінною у EXP та точний збіг молодших біт SHA-256

хешу із фрагментом підпису у Fairoze; обидва механізми руйнуються при зміні меж токенизації та набору tokenів. SEMSTAMP завдяки SBERT-кодуванню речень утримує $F_1 = 0,950$, а KGW у межах $N = 20$ показує 1,000 – частково завдяки якості NLLB-200 на парі англійська – китайська і обмеженому розміру вибірки. Видалення слів дає лише косметичне просідання у діапазоні 0,905–0,976 для всіх алгоритмів, окрім KGW та SegmentWM, що його не помічають. Окремий випадок становить SegmentWM під контекстною заміною синонімів: $F_1 = 0,644$ при $TPR = 0,950$ та $TNR = 0,000$ – тобто всі немарковані тексти після BERT-заміни хибно класифікуються як марковані, що є прямим наслідком агресивного зсуву логітів у стандартній конфігурації MarkLLM. У середньому за чотирма атаками найвищі значення F_1 показують KGW (1,000) та SEMSTAMP (0,982); єдина асиметрична схема Fairoze посідає шосте місце з семи (0,921).

2.2. Розробка алгоритму стеганографічного вбудовування водяного знаку сумісного з технологією блокчейн на основі проведених досліджень

2.2.1. Поточна екосистема blockchain-native автономних агентів на основі стандарту ERC-8004

Стандарт ERC-8004 [5] закріплює уніфікований реєстр ідентичностей автономних програмних агентів у мережі Ethereum та сумісних з нею мережах. Кожна реєстрація агента складається з адреси Ethereum-акаунту (EOA), декларованого набору навичок і сервісних кінцевих точок (endpoints) та посилання на off-chain JSON-метадані з розширеним описом агента; поверх базового реєстру стандарт визначає три додаткових реєстри довіри (validation, reputation, identity bindings).

Принципово важливою для подальшого дизайну є та обставина, що EOA автономного агента є водночас і його криптографічною ідентичністю, і його платіжним інструментом, і ключем доступу до його ж метаданих у ERC-8004. Криптографічно це означає, що кожен агент апіорі володіє парою ключів secp256k1 : приватний ключ sk використовується для підпису транзакцій (фактично як «доказ дії»), а відповідна публічна адреса $A = \text{keccak256}(\text{pk})_{[12:32]}$ є тим самим

первинним ключем у реєстрі ERC-8004. Процедура відновлення публічного ключа з ECDSA-підпису (operation `esrecover`, на віртуальній машині Ethereum реалізована як прекомпільований контракт за адресою 0x01) є вбудованою як у власне віртуальну машину, так і у будь-яку off-chain бібліотеку Web3. Звідси випливає, що верифікатор, який отримав на вхід текст-кандидат і адресу A , апіорно має все необхідне для криптографічної перевірки авторства: реєстр ERC-8004 для отримання параметрів алгоритму, та операцію `esrecover` для перевірки самого підпису. Жодних додаткових публічних ключів зберігати не потрібно – у цьому й полягає природна сумісність `secp256k1`-підписів із моделлю довіри ERC-8004.

2.2.2. Критерії сумісності алгоритму вбудовування водяного знака з блокчейн-екосистемою

З наведеного аналізу екосистеми та з результатів розділів 2–3 можна сформулювати три необхідні умови, яким має задовольняти алгоритм вбудовування водяного знака, щоб бути практично застосовним у середовищі автономних агентів на основі ERC-8004. Ці умови є посиленням загальних критеріїв вступу і вводяться як окремий конструктивний клас – *blockchain-native watermarks*.

Критерій (а) – криптографічна прив'язка до ERC-8004 запису. Алгоритм має забезпечувати, що з валідного маркованого тексту однозначно та обчислювально перевірно відновлюється саме та ідентичність A , яка зареєстрована у ERC-8004 як автор контенту. Формально, для процедури верифікації V та маркованого тексту s , згенерованого агентом з адресою A , має виконуватися $V(s) = A$ з імовірністю, близькою до одиниці; відновлення довільної іншої адреси $A' \neq A$ має бути обчислювально нездійсненним без володіння sk_A . Цей критерій не задовольняється жодним з симетричних алгоритмів розділу 2 (KGW, EXP, Unbiased, SEMSTAMP, SynthID-Text, SegmentWM), оскільки в них виявлення водяного знака не вирізняє конкретного автора, а лише факт використання спільного секретного ключа K .

Критерій (б) – повнота метаданих верифікації. ERC-8004 запис агента має містити у своєму JSON-описі повний набір параметрів, необхідних і достатніх для роботи декодера: ідентифікатор алгоритму та його версію, довжину префікса, що підлягає підпису, схему хешування, параметри сегментного каналу, параметри коду, що виправляє помилки, тощо. Інакше кажучи, верифікатор не повинен знати «зі сторонніх

джерел» жодного параметра алгоритму – отримання адреси A агента з ERC-8004 реєстру має автоматично надавати усі необхідні налаштування декодера.

Критерій (в) – асиметричність схеми. Процедура вбудовування має використовувати приватний матеріал sk (приватний ключ EOA агента), а процедура верифікації – лише публічно доступну інформацію (адресу A , метадані ERC-8004 та згенерований текст). Розкриття sk верифікатору заборонене; зворотно, неможливість підробити маркований текст без знання sk має зводитися до обчислювальної складності задачі ECDSA.

2.2.3. Дизайн алгоритму ChainMark: асиметричне сегментне вбудовування ECDSA-підпису

Запропонований алгоритм, далі позначений як ChainMark, за своєю архітектурою є першим представником класу blockchain-native watermarks. Концептуально він трансформує ідею симетричної верифікації через спільний секрет K , властиву KGW і SegmentWM, в асиметричну верифікацію через ECDSA-підпис: підписати може лише власник приватного ключа sk_A , тоді як верифікувати – будь-який учасник мережі за допомогою операції `esrecover` без знання жодних секретів. Мультибітовий канал SegmentWM використовується як транспортний механізм для самого підпису, а ERC-8004 JSON – як єдине джерело параметрів декодера. Конфігурація вбудовується у ERC-8004 JSON-метадані агента у вигляді мінімального опису алгоритму:

```
{
  "watermark": {
    "algorithm": "ChainMark",
    "model_id": "Qwen/Qwen2.5-Coder-1.5B",
    "tokenizer_id": "Qwen/Qwen2.5-Coder-1.5B",
    "prefix_len": 256,
    "ngram_len": 3,
    "bits_per_segment": 8,
    "k": 64,
    "n": 100,
    "gamma": 0.25,
    "delta": 8.0,
    "hash_scheme": "keccak-256"
  }
}
```

Тут `model_id` – канонічний ідентифікатор мовної моделі-генератора (Hugging Face hub identifier або еквівалентний URI), значення якого додатково входить у підписане повідомлення M і таким чином криптографічно фіксує модель, що згенерувала текст; `tokenizer_id` – ідентифікатор токенизатора, обов'язковий для декодера, оскільки сегментний механізм SegmentWM обчислює хеш від ідентифікатора попереднього токена s_{i-1} і вимагає від декодера побітово ідентичної токенизації тексту-кандидата; у переважній більшості випадків це поле збігається з `model_id`, але виокремлюється як самостійне для випадків спільних або кастомних токенизаторів. `prefix_len = L` – фіксована кількість символів підписуваного префікса відповіді (модельно-незалежна одиниця, що уніфікує визначення межі префікса між різними токенизаторами); `ngram_len = N` – токени довжина контекстного вікна, від якого обчислюється хеш-seed зелених списків. Параметр N детермінує рівномірність розподілу сегментних індексів j_i по позиціях тексту і тим самим – мінімальну довжину фрагмента, потрібну для надійного декодування фіксованого ECDSA-підпису. При $N = 1$ хеш-seed визначається одним попереднім токеном. При $N = 3$ ентропія контекстного кортежу різко зростає, триграми у нормальному тексті майже унікальні, і j_i розподіляються приблизно рівномірно по n сегментах; кожен сегмент отримує $\sim T/n$ голосів і вже за $T \approx 7 \cdot 10^2$ токенів сигнал упевнено перевищує шум. Платою за збільшення N є послаблення робастності: заміна одного токена руйнує контрибуцію наступних N позицій до відповідних сегментів. Для blockchain-native сценарію, де ємність каналу має гарантувати доставку 520 біт підпису у межах однієї відповіді LLM-агента, обирається $N = 3$ як емпіричний компроміс між компактністю каналу та робастністю до локальних редагувань. `bits_per_segment` – кількість бітів інформаційного навантаження на один сегмент; $k = 64$, $n = 100$ – параметри коду Ріда–Соломона $RS(n, k)$, що виправляє до $t = \lfloor (n - k)/2 \rfloor = 18$ помилок; `gamma`, `delta` – параметри зеленого списку, успадковані з SegmentWM [16]; `hash_scheme` – кеccak-256, що збігається з функцією хешування Ethereum.

2.2.3.1. Алгоритм кодування SE

Вхід: запит користувача P , приватний ключ sk_A агента, його адреса A , конфігурація W з ERC-8004. Вихід: маркований стеготекст s' .

1. Згенерувати перші L символів відповіді без втручання у семплування, отримавши префікс s_0 .
2. Побудувати повідомлення для підпису, що включає, окрім префікса, контекстуальну прив'язку до конкретного запиту:

$$M = s_0 \parallel A \parallel \text{model_id} \parallel \tau,$$

де $\parallel$ – конкатенація байтів; A – 20-байтова адреса агента; model_id – фіксована ідентифікаційна стрічка LM, що збігається зі значенням з ERC-8004 JSON; τ – 8-байтова мітка часу; включення A , model_id та τ запобігає replay-атакам, у яких зловмисник міг би скопіювати валідний підпис з однієї відповіді у іншу.

3. Обчислити хеш повідомлення:

$$h = \text{keccak256}(M) \in \{0, 1\}^{256}$$

4. Сформувати ECDSA-підпис на кривій secp256k1 :

$$\sigma = \text{Sign}_{sk_A}(h) = (r, s, v)$$

де $r, s \in \mathbb{Z}_q$ – стандартні компоненти ECDSA, $v \in \{0, 1\}$ – біт відновлення, що дозволяє однозначно реконструювати публічний ключ з підпису. Серіалізація σ у фіксовані 65 байт = 520 біт є канонічною для Ethereum.

5. Закодувати σ кодом Ріда–Соломона $RS(n, k)$ над $\text{GF}(2^{b/k})$, де $b/k = \text{bits_per_segment}$:
6. $\sigma^{\text{cw}} = \text{RS-encode}(\sigma) \in (\{0, 1\}^{b/k})^n$
7. При $b/k = 8, k = 64, n = 100$ розмір інформаційного навантаження становить $64 \cdot 8 = 512$ біт; компоненти (r, s) підпису (64 байти = 512 біт) розташовуються у цій смузі, тоді як біт v брутфорситься верифікатором. Решта $(n - k) \cdot (b/k) = 288$ біт використовуються кодом для виправлення до $t = 18$ символьних помилок, що відповідає робастності, зазначеній у [16].
8. Продовжити генерацію тексту, починаючи з позиції L , із застосуванням сегментного механізму SegmentWM: для кожного нового токена i хеш від попереднього токена s_{i-1} детермінує номер сегмента $j_i \in \{0, \dots, n - 1\}$, на який «припадає» цей токен; хеш пари $(s_{i-1}, \sigma_{j_i}^{\text{cw}})$ задає псевдовипадковий «зелений» список $G(s_{i-1}, \sigma_{j_i}^{\text{cw}})$, до логітів якого додається зсув δ .

9. Генерація триває до досягнення статистично достатньої довжини $T \geq T_{\min}$, за якої кожен з n сегментів отримує мінімальну кількість токенів-носіїв, потрібну для надійного декодування. Для $\gamma = 0,25$, $\delta = 8,0$, $N = 3$ це відповідає $T_{\min} \approx 7n = 700$ токенам після префікса; зменшення N до 1 збільшує цю величину приблизно на порядок (до $\sim 6 \cdot 10^3$ токенів) через нерівномірність розподілу сегментних індексів.

2.2.3.2. Алгоритм декодування SD

Вхід: текст-кандидат s' ; адреса агента A , від імені якого нібито створено s' . Вихід: булеве рішення $\{\text{True}, \text{False}\}$ – чи дійсно s' створено агентом A .

1. Виконати on-chain (або через локальне індексування) запит до ERC-8004 реєстру за адресою A та отримати JSON-метадані W . Зчитати з W параметри `prefix_len`, `bits_per_segment`, `k`, `n`, `gamma`, `delta`, `hash_scheme`. Якщо у W відсутнє поле `watermark` або не підтримується значення `algorithm`, верифікація завершується відмовою.
2. Розщепити $s' = s'_0 \parallel s'_1$, де $|s'_0| = L$ (перші `prefix_len` символів кандидата).
3. Застосувати декодер SegmentWM до s'_1 для відновлення кодового слова: для кожного сегмента j перебрати усі $2^{b/k}$ значень і обрати таке, що максимізує кількість «зелених» токенів.
4. Виконати декодування коду Ріда–Соломона з виправленням до t помилок.
5. Якщо число помилок перевищує t , повернути False.
6. Реконструювати повідомлення для перевірки підпису:

$$\hat{M} = s'_0 \parallel A \parallel \text{modelId} \parallel \tau$$
де `modelId` читається з W , а τ маркування часу.
7. Обчислити $\hat{h} = \text{keccak256}(\hat{M})$ та виконати відновлення публічного ключа за допомогою прекомпіляції `esrecover`:
8. $\hat{A} = \text{esrecover}(\hat{h}, \hat{\sigma})$
9. Повернути True тоді й лише тоді, коли $\hat{A} = A$. У разі невизначеності біта відновлення v – спробувати обидва значення та повернути True, якщо хоча б одне дає збіг.

2.2.3.3. Властивості та сумісність із критеріями

Запропонована конструкція задовольняє усім трьом критеріям підрозділу 2.2.2 за побудовою та частково задовольняє критеріям визначеним у вступі:

1. Асиметричність (критерій в). Підпис σ можна сформувати лише за наявності sk_A , оскільки задача підробки ECDSA на secp256k1 без приватного ключа є математично нерозв'язною за наявних комп'ютерних ресурсів. Усі компоненти декодера використовують виключно публічну адресу A , публічні параметри з ERC-8004 та публічно доступну операцію `esrecover`. Жоден секретний матеріал не розкривається.
2. Прив'язка до ERC-8004 запису (критерій а). Результат декодування ($\hat{A}$) збігається з адресою A агента у реєстрі тоді й лише тоді, коли σ було сформовано саме ключем sk_A . Підробка з адресою $A' \neq A$ зведеться до знаходження колізії в `esrecover`, що еквівалентно зламу ECDSA. Таким чином, ChainMark забезпечує не просто факт маркування, а саме доказ авторства конкретного агента, зареєстрованого у мережі.
3. Повнота метаданих (критерій б). Усі сім параметрів, потрібних для роботи декодера, пакуються у мінімальний JSON-фрагмент, що додається до стандартного агентського запису ERC-8004. Запит до реєстру за адресою A повертає верифікатору повний контекст алгоритму без будь-якого додаткового узгодження.
4. Робастність та швидкість декодування. Завдяки сегментному коду складність декодування становить $O(n \cdot 2^{b/k}) = O(100 \cdot 256) = 2,56 \cdot 10^4$ операцій, що відповідає характеру SegmentWM і, згідно з [16], дозволяє витягувати 32-бітне навантаження за частки секунди – на відміну від $O(2^{520})$ повного перебору, що був би потрібен для прямолінійної реалізації Fairgoze з еквівалентною довжиною підпису. Код $RS(100, 64)$ виправляє до $t = 18$ символічних помилок, що нестандартно навіть для типових атак парафразування.
5. Робастність до replay-атак. Включення A , `model_id` та τ у підписане повідомлення M запобігає тривіальному копіюванню валідного підпису з однієї відповіді у іншу: будь-яка зміна цих компонентів змінює h і робить підпис недійсним. Мітка часу τ також дозволяє

верифікатору відсікати застарілі (і потенційно скомпрометовані) відповіді шляхом задання вікна допустимої свіжості.

6. Бюджет довжини. Мінімальна довжина маркованого фрагмента визначається сумою префікса L символів та смуги вбудовування $T_{\min}$ токенів. За вибраних параметрів $L = 256$, $N = 3$ мінімальна довжина становить $T_{\min} \approx 700$ що приблизно втричі менше за мінімальну довжину Fairuze і відповідає типовому обсягу однієї змістовної відповіді LLM-агента у режимі обслуговування завдання.

ВИСНОВКИ

У роботі здійснено огляд методів стеганографічного вбудовування цифрових водяних знаків у текст великих мовних моделей та розроблено дизайн власного алгоритму, сумісного з блокчейн-екосистемою автономних агентів. Розглянуто сім сучасних алгоритмів – KGW, EXP, Unbiased, SEMSTAMP, SynthID-Text, SegmentWM та Fairoze – і проведено їх експериментальне порівняння на корпусі C4 з моделлю Qwen2.5-Coder-1.5B. Встановлено, що при ідентичній якості детектування на чистому тексті ($F_1 = 1,000$) алгоритми різняться за часом генерації (164–1187 с), перплексією (5,25–67,81) та середнім F_1 під атаками (0,898–1,000), причому єдина асиметрична схема Fairoze посідає шосте місце з семи за робастністю – що кількісно засвідчує розрив між сучасним станом галузі та потребами публічної верифікації авторства. На основі аналізу ERC-8004 сформульовано три критерії сумісності – криптографічна прив'язка до on-chain ідентичності, повнота метаданих верифікації, асиметричність схеми – та запропоновано перший алгоритм, що задовольняє усім трьом: ChainMark. Він трансформує симетричну ідею спільного секрета K в асиметричну ідею ECDSA-підпису EOA-ключем агента, використовує мультибітовий канал SegmentWM як транспорт для серіалізованого підпису, а ERC-8004 JSON – як єдине джерело параметрів декодера. Тим самим запропоновано перший представник класу blockchain-native watermarks. Експериментальна валідація ChainMark становить предмет подальшого практичного дослідження.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. BuzzBD AI: Autonomous Zero-Human Company Platform [Електронний ресурс]. – Режим доступу: <https://buzzbd.ai/> (дата звернення: 29.04.2026).
2. AgentaOS: Operating System for Autonomous AI Agents [Електронний ресурс]. – Режим доступу: <https://agentaos.ai/> (дата звернення: 29.04.2026).
3. 8004 Scan: ERC-8004 Agent Registry Explorer [Електронний ресурс]. – Режим доступу: <https://8004scan.io/> (дата звернення: 29.04.2026).
4. The Spawn Network. World [Електронний ресурс]. – Режим доступу: <https://thespawn.io/world> (дата звернення: 29.04.2026).
5. ERC-8004: Trustless Agents Registry Standard [Електронний ресурс] // Ethereum Improvement Proposals. – Режим доступу: <https://eips.ethereum.org/EIPS/eip-8004> (дата звернення: 29.04.2026).
6. Agent Commerce Protocol (ACP) Documentation [Електронний ресурс]. – Режим доступу: <https://www.agenticcommerce.dev> (дата звернення: 29.04.2026).
7. x402: Internet-Native Payments Protocol [Електронний ресурс]. – Режим доступу: <https://www.x402.org/> (дата звернення: 29.04.2026).
8. Hopper N. J., Langford J., von Ahn L. Provably Secure Steganography : Technical Report CMU-CS-02-149 [Електронний ресурс] / School of Computer Science, Carnegie Mellon University. – Pittsburgh, 2002. – 18 p. – Режим доступу: <https://eprint.iacr.org/2002/137> (дата звернення: 29.04.2026).
9. Judge J. C. Steganography: Past, Present, Future : SANS Institute Information Security Reading Room White Paper [Електронний ресурс]. – SANS Institute, 2001. – 28 p. – Режим доступу: <https://www.sans.org/white-papers/552/> (дата звернення: 29.04.2026).
10. Pan L., Liu A., He Z. et al. MarkLLM: An Open-Source Toolkit for LLM Watermarking [Електронний ресурс] // Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. – 2024. – Режим доступу: <https://arxiv.org/abs/2405.10051> (дата звернення: 29.04.2026).
11. Kirchenbauer J., Geiping J., Wen Y., Katz J., Miers I., Goldstein T. A Watermark for Large Language Models [Електронний ресурс] // Proceedings

of the 40th International Conference on Machine Learning (ICML). – 2023. – Режим доступа: <https://arxiv.org/abs/2301.10226> (дата звернення: 29.04.2026).

12. Aaronson S. Watermarking of Large Language Models [Електронний ресурс] : Talk at the Simons Institute for the Theory of Computing, University of California, Berkeley. – 17 серпня 2023. – Режим доступа: <https://simons.berkeley.edu/talks/scott-aaronson-ut-austin-openai-2023-08-17> (дата звернення: 29.04.2026).

13. Hu Z., Chen L., Wu X. et al. Unbiased Watermark for Large Language Models [Електронний ресурс] // International Conference on Learning Representations (ICLR). – 2024. – Режим доступа: <https://arxiv.org/abs/2310.10669> (дата звернення: 29.04.2026).

14. Hou A. B., Zhang J., He T., Wang Y., Chuang Y.-S., Wang H., Shen L., Van Durme B., Khashabi D., Tsvetkov Y. SemStamp: A Semantic Watermark with Paraphrastic Robustness for Text Generation [Електронний ресурс] // Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL). – 2024. – Режим доступа: <https://arxiv.org/abs/2310.03991> (дата звернення: 29.04.2026).

15. Dathathri S., See A., Ghaisas S. et al. Scalable Watermarking for Identifying Large Language Model Outputs // Nature. – 2024. – Vol. 634. – P. 818–823. – DOI: 10.1038/s41586-024-08025-4.

16. Qu W., Zheng W., Tao T., Yin D., Jiang Y., Tian Z., Zou W., Jia J., Zhang J. Provably Robust Multi-bit Watermarking for AI-generated Text [Електронний ресурс]. – 2024. – Режим доступа: <https://arxiv.org/abs/2401.16820> (дата звернення: 29.04.2026).

17. Fairoze J., Garg S., Jha S., Mahlouiifar S., Mahmood M., Wang M. Publicly-Detectable Watermarking for Language Models [Електронний ресурс]. – 2023. – Режим доступа: <https://arxiv.org/abs/2310.18491> (дата звернення: 29.04.2026).

18. Liu Y., Ott M., Goyal N., Du J., Joshi M., Chen D., Levy O., Lewis M., Zettlemoyer L., Stoyanov V. RoBERTa: A Robustly Optimized BERT Pretraining Approach [Електронний ресурс]. – 2019. – Режим доступа: <https://arxiv.org/abs/1907.11692> (дата звернення: 29.04.2026).

19. Damodaran P. Parrot: Paraphrase Generation for NLU [Електронний ресурс]. – 2021. – Режим доступа: https://huggingface.co/prithivida/parrot_paraphraser_on_T5 (дата звернення: 29.04.2026).

20. NLLB Team, Costa-jussà M. R., Cross J., Çelebi O., Elbayad M., Heafield K., Heffernan K., Kalbassi E., Lam J., Licht D., Maillard J. et al. No Language Left Behind: Scaling Human-Centered Machine Translation [Електронний ресурс]. – 2022. – Режим доступу: <https://arxiv.org/abs/2207.04672> (дата звернення: 29.04.2026).

21. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [Електронний ресурс] // Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT). – 2019. – Режим доступу: <https://arxiv.org/abs/1810.04805> (дата звернення: 29.04.2026).

22. MarkLLM (модифікована версія): референсна реалізація алгоритму ChainMark та експериментальний код курсової роботи [Електронний ресурс] : GitHub repository. – 2026. – Режим доступу: <https://github.com/YaTut1901/MarkLLM> (дата звернення: 29.04.2026).